Foundations of Computational Linguistics - Lecture 03

Lecture overview

Lecture 03 introduces foundational concepts in computational linguistics, beginning with the challenges of text preprocessing such as tokenization and sentence detection. The lecture explains that computers cannot inherently distinguish words from characters, necessitating explicit tokenization to define word boundaries, a process complicated by apostrophes, hyphens, compound words, and multi-word proper names like "San Francisco." It contrasts traditional whole-word approaches with modern subword units used in neural models. The session then transitions to language models within the statistical paradigm, which estimate the probability of well-formed sentences by decomposing joint probabilities into conditional n-gram sequences based on limited prior context. These models are applied to tasks including speech recognition, machine translation, and completion prediction, where they distinguish between probable and improbable word sequences.

The second half of the lecture focuses on sequence labeling tasks like Part-of-Speech tagging and Named Entity Recognition (NER), utilizing IOB encoding to classify entities such as persons, locations, and organizations. It details the manual definition of features, including context words and character substrings, and discusses global optimization techniques to ensure the best overall label sequence rather than just locally optimal choices. Finally, the lecture establishes rigorous standards for experimental evaluation, emphasizing the necessity of splitting data into training, development, and test sets to prevent data leakage. It covers the importance of baselines, upper bounds defined by human agreement, and metrics beyond accuracy-such as precision, recall, F1-measure, and micro/macro-averaging-to properly assess model performance on imbalanced datasets or sequence tasks.

Slide 1: faculty of arts

This lecture, titled "Language models and word meaning," is the third session in the Computational Linguistics course offered by the Faculty of Arts. It is presented by Tim Van de Cruys on Tuesday, October 6, 2026. The slide serves as the title page introducing the specific topic and instructor for this academic session.

Slide 2: Tokenization

Computers encode text as a continuous sequence of characters where words, spaces, and punctuation are treated identically, meaning the system cannot inherently distinguish actual words. To utilize words as features for classification, we must explicitly define and mark word boundaries through a process called tokenization. This step identifies specific character ranges as distinct words, separating them from surrounding spaces or punctuation.

Slide 3: Tokenization

Tokenization addresses the fundamental question of defining a word as a consecutive sequence of characters separated by whitespace or punctuation. The slide highlights the specific challenge posed by apostrophes, illustrating how they complicate segmentation in cases like "sms'en," "gsm'etje," and "doin'." This raises the critical issue of whether such internal punctuation should split tokens or remain part of a single word unit.

Slide 4: Tokenization

Tokenization defines words as consecutive character sequences separated by whitespace or punctuation, though this approach is not always straightforward. Ambiguities arise with apostrophes in contractions like "doin'" or Dutch forms like "sms'en," where separating the punctuation may be undesirable. Similarly, hyphenated terms such as "self-driving" challenge standard separation rules, requiring careful decisions on whether to treat them as single tokens or multiple words.

Slide 5: Tokenization

Tokenization defines a word as a character sequence separated by whitespace or punctuation, yet this rule fails for apostrophes, hyphens, and languages lacking spaces. Challenges arise with compound words like "langeafstandsloper," Chinese scripts without whitespace, and agglutinating languages such as Turkish or Basque. In these cases, multiple morphemes chain into a single word to express concepts that require several separate words in English.

Slide 6: Tokenization

Tokenization involves determining what constitutes a word within a text. A key challenge in this process is handling proper names, such as "San Francisco" and "New York," which consist of multiple tokens but represent single entities. This slide highlights the ambiguity inherent in defining word boundaries for multi-word proper nouns.

Slide 7: Tokenization

Tokenization requires deliberate decisions about what constitutes a word, as the process is not merely splitting text by whitespace or punctuation. Key challenges include determining whether proper names like "San Francisco" should be treated as single tokens or separate ones, and how to handle fixed expressions such as "by and large" or "piece of cake." These examples illustrate that defining tokens involves reflecting on linguistic units rather than relying solely on simple delimiters.

Slide 8: Tokenization

Tokenization identifies words within text, addressing challenges like proper names (e.g., "San Francisco") and fixed expressions (e.g., "by and large"). While statistical paradigms rely on defining whole words for features, modern neural models avoid this question by splitting text into frequency-based subword units, such as byte pair encoding. These subword units are often smaller than actual words and will be discussed further later in the course.

Slide 9: Sentence detection

Sentence detection relies on unambiguous punctuation like question and exclamation marks, whereas dots are highly ambiguous due to abbreviations such as "U.N." or "etc." As illustrated by examples involving the U.N. General Assembly and musical genres, a dot does not always signal a sentence boundary. Therefore, determining where a sentence ends requires analyzing the surrounding context rather than relying solely on individual punctuation marks.

Slide 10: Language models

Language models within the statistical paradigm estimate the probability of a string being well-formed, offering a probabilistic alternative to binary symbolic rule-based models. These models determine grammaticality and generate sentences by specifying a probability distribution where the probabilities of all possible sentences in a language sum to one. This approach is widely used for tasks like predicting the next word in a sequence.

Slide 11: Uses of language models

Language models enhance speech recognition, OCR, handwriting recognition, machine translation, generation, and context-sensitive spelling correction by selecting the most probable sentences. For instance, a model distinguishes "I ate a tomato" from the unlikely phonetic sequence "eye eight atom eight oh." Similarly, it identifies errors in sequences like "Their is a problem wit this sentence" where individual words are valid but the overall context is incorrect.

Slide 12: Completion prediction

Completion prediction refers to a language model's ability to predict the next word in a sentence, as demonstrated by examples like "Please turn off your mobile ____" and "If he loses, he has only himself to ____." This concept underpins predictive text input systems, which guess what a user is typing and provide choices to complete it.

Slide 13: Statistical language model

A statistical language model computes sentence probability by decomposing the problem into estimating individual word probabilities given limited prior context, known as an n-gram model. Instead of counting entire sentences directly, which is often impractical, the model calculates the probability of each word based on preceding words and multiplies these values to determine the final sentence probability.

Slide 14: Statistical language model

A statistical language model computes sentence probability by dividing the count of a specific sentence in a corpus by the total number of sentences. For example, calculating the probability of "the dog was running across the green field" using the entire internet yields zero because this exact sequence never appears. This result highlights a limitation: while the sentence is perfectly understandable to humans, its statistical probability is zero due to data sparsity in large corpora.

Slide 15: Statistical language model

To compute the joint probability of a sentence like "the dog was running across the green field," statistical language models decompose it into a product of conditional probabilities. This process calculates the probability of each word given all preceding words, starting with p(the) and continuing sequentially through p(field|the dog was running across the green). This decomposition transforms the complex joint probability problem into manageable conditional components.

Slide 16: Statistical language model

To compute the joint probability of a sentence like "the dog was running across the green field," statistical language models decompose it into a chain of conditional probabilities. Because exact long-context sequences often have zero probability in corpora, the model uses only a limited context, specifically considering just the two preceding words to estimate each subsequent word's probability effectively.

Slide 17: Statistical language model

Statistical language models compute the joint probability of a word sequence by decomposing it into a chain of conditional probabilities. For the sentence "the dog was running across the green field," this involves calculating terms like p(dog|the) and p(was|the dog). The model relies on using only a limited context to estimate these probabilities efficiently.

Slide 18: Estimating probabilities

Probabilities are estimated by counting occurrences within a sufficiently large corpus. The probability of a word like "the" is calculated as its count divided by the total word count, while conditional probabilities, such as "dog" given "the," are derived by dividing the co-occurrence count by the count of the preceding context. This method extends to longer contexts, where the probability of a word depends on the frequency of the specific sequence it follows.

Slide 19: N-gram models

N-gram models estimate the probability of a word given limited prior context, where an n-gram uses exactly n-1 preceding words. These models rely on the Markov assumption, which posits that future behavior depends only on recent history; consequently, an n-gram model functions as an (n-1)th-order Markov model. Sentence probability is calculated by multiplying individual word probabilities, with examples ranging from unigrams (single words) to trigrams (three-word sequences).

Slide 20: Part of speech

Part of speech categorizes words into classes based on their similar behavior within sentences, providing crucial context about word placement. For instance, adjectives are typically succeeded by nouns, while personal pronouns are often followed by verbs. This contextual information supports language processing applications through part-of-speech tagging, which automatically assigns these categories to individual words in a sequence.

Slide 21: Named entity recognition

Named entity recognition is an important NLP subtask that finds and classifies names within text. To perform this classification effectively, systems utilize specific features beyond just the word itself, such as punctuation marks like full stops or substrings containing hyphens. Additionally, word forms are critical indicators, particularly patterns involving capital letters followed by specific character sequences, which help identify the correct named entity tag.

Slide 22: Named entity recognition

Named entity recognition is an important subtask in natural language processing that involves finding and classifying names within a text. The slide illustrates this concept using a sentence about Eurostat, where entities such as the organization "Eurostat," the date "Wednesday 28 October," and locations "Paris" and "Berlin" would be identified and categorized.

Slide 23: Named entity recognition

Named entity recognition is an important NLP subtask that involves finding and classifying proper names in a text into categories such as person, location, date, or organization. This process enables large-scale analysis of entities within texts, which is particularly useful for applications in digital humanities and history. While the example sentence contains locations like Paris and Berlin, dates, and organizations like Eurostat, it notably lacks any person entities.

Slide 24: Named entity recognition

Named entity recognition transforms text classification into a sequence labeling task using IOB encoding, which stands for Inside, Outside, and Beginning. This scheme distinguishes the first token of an entity (e.g., B-ORG for "Eurostat") from subsequent tokens within the same entity (e.g., I-DAT for "28" and "October"), while marking non-entity words as Outside. The distinction between Beginning and Inside tags is essential to correctly separate adjacent entities of the same type.

Slide 25: Manual definition of features

Manual feature definition involves selecting specific attributes such as the current word, surrounding context words, character substrings, and word forms. It also incorporates additional linguistic annotations like Part-of-Speech (POS) tags and context labels from previous or next tokens. These features are extracted from text sequences to support computational linguistic analysis.

Slide 26: Manual definition of features

Manual feature definition involves specifying inputs such as the current word, surrounding context words, character substrings, and word forms. It also incorporates additional linguistic annotations like Part-of-Speech (POS) tags and context labels for previous or next tokens. These features are applied to text sequences, exemplified by the sentence "Paramedics treat woman and boy after crash in Stoke-on-Trent."

Slide 27: Manual definition of features

Manual feature definition involves selecting specific attributes such as the current word, surrounding context words, character substrings, and word forms. It also incorporates additional linguistic annotations like Part-of-Speech (POS) tags and context labels from previous or next tokens. These features are extracted from text sequences to support computational linguistic analysis.

Slide 28: Manual definition of features

Manual feature definition involves selecting specific attributes such as the current word, surrounding context words, character substrings, and word forms. It also incorporates additional linguistic annotations like Part-of-Speech (POS) tags and context labels from previous or next tokens. These features are extracted from text sequences to support computational linguistic analysis.

Slide 29: Manual definition of features

Manual feature definition involves selecting specific attributes such as the current word, surrounding context words, character substrings, and word forms. It also incorporates additional linguistic annotations like Part-of-Speech (POS) tags and context labels from previous or next tokens. These features are applied to text sequences, exemplified by the sentence "Paramedics treat woman and boy after crash in Xx-x-Xx."

Slide 30: Manual definition of features

Manual feature definition involves selecting specific attributes such as the current word, surrounding context words, character substrings, and word forms. It also incorporates additional linguistic annotations like Part-of-Speech (POS) tags and context labels for previous or next tokens. The slide illustrates these concepts using a sentence example where words are paired with their corresponding POS tags to demonstrate feature extraction.

Slide 31: Manual definition of features

Manual feature definition involves selecting specific attributes such as the current word, surrounding context words, character substrings, and word forms. It also incorporates additional linguistic annotations like Part-of-Speech (POS) tags and context labels for previous or next tokens. The slide illustrates this approach using a sentence example where words are paired with their corresponding POS tags and label sequences.

Slide 32: Manual definition of features

Manual feature definition involves selecting specific attributes such as the current word, surrounding context words, character substrings, and word forms. It also incorporates additional linguistic annotations like Part-of-Speech (POS) tags and context labels for previous or next tokens. The slide illustrates this approach using a sentence example where words are paired with their corresponding POS tags and entity labels to demonstrate feature extraction.

Slide 33: Sequence tagging

Sequence tagging treats labeling each word in a sequence as a simple classification task performed sequentially. To predict the best label for the current word, the model utilizes observed features from the current, previous, and future context, alongside predicted labels from preceding words. This approach relies on feature engineering to construct informative inputs, such as part-of-speech tags or word shapes, rather than relying solely on infrequent raw words.

Slide 34: Global optimization

Global optimization addresses the limitation of sequence prediction where a series of local judgments ignores global optimality, potentially missing better overall combinations. To solve this, instead of emitting only the single most probable answer for each word, the system predicts the top n best answers. By retaining these highly probable options, the method computes the globally optimal sequence by evaluating all items and their probabilities together at the end.

Slide 35: Experiment

Experiments are a common and intrinsic procedure in NLP used to determine which model performs best for a specific task, such as sentiment classification. This process involves comparing three distinct classifiers: a symbolic approach using hand-written rules (e.g., mapping "cool" to positive), Naive Bayes based on probability counting, and logistic regression utilizing feature weights and normalization. By testing these models against one another, researchers can identify the most effective solution for their particular application.

Slide 36: Train, development, test

Data must be split into training, development, and test sets to ensure models generalize rather than memorize. The training set estimates parameters, while the development set guides choices regarding features, hyperparameters, and model selection. The test set is reserved exclusively for a single final evaluation to report performance, preventing data leakage that could invalidate results.

Slide 37: Leakage

Leakage occurs when information from the test set inadvertently enters the model, resulting in an overly optimistic reported score. This often happens unintentionally through practices like selecting hyperparameters based on test performance, computing vocabulary before data splitting, or including duplicate reviews and texts by the same author in both training and test sets. To prevent this, all learning and selection processes must be conducted without any access to the test set.

Slide 38: Baseline

A model's performance score only has meaning when compared against a baseline, which represents the simplest possible approach such as predicting the majority class or using simple rules. For instance, in toxic comment detection where 95% of comments are non-toxic, a trivial model that always predicts "non-toxic" achieves 95% accuracy despite failing to identify any actual toxic content. Consequently, if a sophisticated model fails to outperform this simple baseline, the result is considered negative.

Slide 39: Upper bound

The upper bound represents the best expected score, typically defined by human performance measured through inter-annotator agreement. This metric calculates how often independent annotators agree on the same items, using measures like percentage agreement or Krippendorff's alpha to correct for chance. Consequently, a model's score is interpreted relative to this limit, which varies by task; for instance, sentiment analysis shows high agreement (80-90%), whereas subjective tasks like toxicity detection yield considerably lower bounds.

Slide 40: Beyond accuracy

Beyond accuracy, the confusion matrix defines four outcomes: true positives, false positives, false negatives, and true negatives based on gold standard labels. Precision measures the correctness of positive predictions, while recall assesses how many actual positives were identified; these are balanced by the F1-measure, which calculates their harmonic mean. Relying solely on accuracy can be misleading, as a baseline predicting "not toxic" for all items might achieve 95% accuracy yet yield 0% recall and F1 for the toxic class.

Slide 41: Multiple classes

For multi-class classification, compute precision, recall, and F1 per class before averaging. Micro-averaging pools all decisions into a single result, making it dominated by frequent classes, while macro-averaging calculates the mean of per-class scores so every class counts equally. These two methods can yield considerably different results when dealing with imbalanced datasets, such as topic classification where some categories like "culture" have far fewer instances than others.

Slide 42: Performance on sequences

Evaluating named entity recognition requires computing precision, recall, and F-measure over entities rather than individual words to avoid misleadingly high accuracy from the dominant 'outside' label. An entity is considered correct only if both its boundaries and type match the test set; for instance, tagging "Michel" instead of "Charles Michel" or mislabeling "European Council" as LOC are incorrect. These metrics balance the percentage of correctly returned entities against the total found in the gold standard.

Slide 43: Thresholds

Naive Bayes and logistic regression output probabilities, requiring a decision threshold (e.g., classifying as toxic if $p > 0.5$) to make predictions. This threshold is a hyperparameter tuned on the development set, where lowering it increases recall but decreases precision, while raising it increases precision but decreases recall. The optimal cutoff is determined by evaluating these trade-offs against computed precision and recall metrics.

Slide 44: Significance

A small accuracy difference between models, such as 84.2% versus 84.7%, is not necessarily significant because test sets are samples and random components like initialization affect results. To ensure reliability, researchers should use sufficiently large test sets, report their size, and repeat experiments with different seeds to calculate means and standard deviations. When differences are small, statistical significance tests like McNemar’s test or paired bootstrap are required to determine if the improvement is genuine rather than due to chance.

Slide 45: Error analysis

While evaluation scores quantify mistake frequency without explaining causes, error analysis involves inspecting a sample of errors from the development set to identify patterns. By categorizing these errors, practitioners can determine frequent issues and potential improvements, often leveraging linguistic knowledge to explain why specific phenomena trigger model failures. This process reveals generalizations in model behavior that quantitative metrics alone cannot provide.

Slide 46: Checklist

This slide presents an eight-point checklist for rigorous experimental evaluation in computational linguistics. Key requirements include maintaining a separate test set used only once, avoiding data leakage, comparing against baselines, and selecting metrics appropriate for the task and class distribution. Additionally, researchers must make all model choices on the development set, report result uncertainty, analyze errors, and support every claim with empirical results.

Key points to remember

The second half of the lecture focuses on sequence labeling tasks like Part-of-Speech tagging and Named Entity Recognition (NER), utilizing IOB encoding to classify entities such as persons, locations, and organizations. It details the manual definition of features, including context words and character substrings, and discusses global optimization techniques to ensure the best overall label sequence rather than just locally optimal choices. Finally, the lecture establishes rigorous standards for experimental evaluation, emphasizing the necessity of splitting data into training, development, and test sets to prevent data leakage. It covers the importance of baselines, upper bounds defined by human agreement, and metrics beyond accuracy-such as precision, recall, F1-measure, and micro/macro-averaging-to properly assess model performance on imbalanced datasets or sequence tasks.